

HANDS ON PRACTICE ON HASHMAP

DAY 70
18/09/25

Hash Map is implemented By Map Interface

```
package DSA.HashMap;

import java.util.*;

public class HashmapDemo {

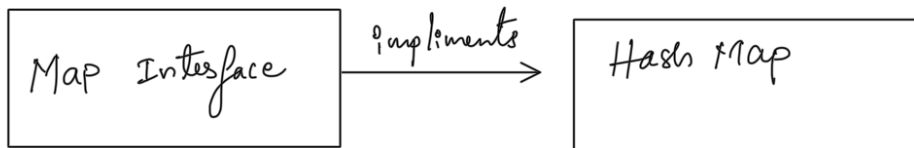
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {

        Map<Integer,String> students = new HashMap<>();

        students.put(key:1,value:"Veer");

    }
}
```

```
interface Map<K, V> {
    interface Map.Entry<K, V> { /* removed */ }
    int size();
    boolean isEmpty();
    boolean containsKey(Object);
    boolean containsValue(Object);
    V get(Object);
    V put(K, V);
    V remove(Object);
    void putAll(Map<? extends K, ? extends V>);
    void clear();
    Set<K> keySet();
    Collection<V> values();
    Set<Map.Entry<K, V>> entrySet();
}
```



```
public class HashmapDemo {

    Run | Debug | Run main | Debug main
    public static void main(String[] args) {

        Map<Integer,String> students = new HashMap<>();

        students.put(key:1,value:"Veer");

    }
}
```

String java.util.Map.put(Integer key, String value)

Associates the specified value with the specified key in this map (optional operation). If the map previously contained a mapping for the key, the old value is replaced by the specified value. (A map *m* is said to contain a mapping for a key *k* if and only if *m.containsKey(k)* would return *true*.)

- Parameters:
 - key key with which the specified value is to be associated
 - value value to be associated with the specified key
- Returns:
 - the previous value associated with *key*, or *null* if there was no mapping for *key*. (A *null* return can also indicate that the map previously associated *null* with *key*, if the implementation supports *null* values.)
- Throws:

Map.put method used to add <key value> to the Hash map

• put (K key, V value)
is used to put the Value to the Hashmap

adding K V to the Students Hash Map

Here key is of type Integer
Value is of type String

The Hash Map stores Key Value Pairs Accordingly

```
DSA > HashMap > J HashmapDemo.java > Java > HashmapDemo > main(String[] args)
5 public class HashmapDemo {
6     public static void main(String[] args) {
7
8         Map<Integer,String> students = new HashMap<>();
9
10        // add Elements To the HashMap Using Put Method
11        students.put(key:1,value:"Veer");
12        students.put(key:2,value:"Karan");
13        students.put(key:3,value:"Rudresh");
14        students.put(key:300,value:"Roshan");
15        students.put(key:25,value:"Nandish");
16        students.put(key:62,value:"Siddu");
17
18        System.out.println();
19
20        System.out.println("Students HashMap Consists of Students ");
21        System.out.println(students);
22
23        System.out.println();
24    }
25 }
26
```

OUTPUT 63 DEBUG CONSOLE PORTS TERMINAL

```
(base) kveeresh@S-MacBook-Air-2 ComputerScience % cd /Users/kveeresh/Desktop/Co
ry/Java/JavaVirtualMachines/openjdk-24.0.1/Contents/Home/bin/java -agentlib:jdwp=
host:57453 --enable-preview -XX:+ShowCodeDetailsInExceptionMessages -cp /Users/kv
eesh/Desktop/2ecbf2b350995b328b049bd629e6946/redhat.java/jdt_ws/ComputerScience_4d
Students HashMap Consists of Students
{1=Veer, 2=Karan, 3=Rudresh, 4=Roshan, 5=Nandish, 6=Siddu}
```

// updating the value

```
25
26     System.out.println(x:"Updating Value For a Key 2");
27     System.out.println(x:"Before Updating: ");
28     System.out.println(students);
29     System.out.println(x:"After Updating: ");
30
31     students.put(key:2, value:"Karan Shrigiri");
32     System.out.println(students);
33
34 }
35
36
```

OUTPUT 63 DEBUG CONSOLE PORTS TERMINAL

Updating Value For a Key 2
Before Updating:
{1=Veer, 2=Karan, 3=Rudresh, 4=Roshan, 5=Nandish, 6=Siddu}
After Updating:
{1=Veer, 2=Karan Shrigiri, 3=Rudresh, 4=Roshan, 5=Nandish, 6=Siddu}

updating student value
for key "2"

Returns true if key
Exists
Returns false if key
Doesn't Exist.

Searching Key

```
35
36 //search
37
38 System.out.println(x:"Searching Operation");
39 int searchkey = 25;
40
41 if(students.containsKey(searchkey)){
42     System.out.println("Students Map Contains key : "+ searchkey );
43 }
44 else{
45     System.out.println("Students Map Doesn't Contains key : "+ searchkey );
46 }
47
48
49
50
```

returns Boolean
value

```
Host:57573 --enable-preview --XX:+ShowCodeDetailsInExceptionMessages -cp /Users/kveeresh/Library/Application\ Support/Code/User/workspaceStorage/2ecbf2b3509965b320b049bd629e6946/redhat.java/jdt_ws/Computer-Science_4e28d2eb/bin DSA.HashMap.HashMapDemo

Students HashMap Consists of Students
{1=Veer, 2=Karan, 3=Rudresh, 25=Nandish, 300=Roshan, 62=Siddu}

Updating Value For a Key 2
Before Updating:
{1=Veer, 2=Karan, 3=Rudresh, 25=Nandish, 300=Roshan, 62=Siddu}
After Updating:
{1=Veer, 2=Karan Shrigiri, 3=Rudresh, 25=Nandish, 300=Roshan, 62=Siddu}
Searching Operation
Students Map Contains key : 25
(base) kveeresh@KS-MacBook-Air-2 Computer-Science %
```

```
//update
System.out.p boolean containsKey(Object)
System.out.p Returns true if this map contains a mapping for the specified key.
System.out.p boolean java.util.Map.containsKey(Object key)
System.out.p Returns true if this map contains a mapping for the specified key. More formally, returns true if and only if this map contains a mapping for a key k such that Objects.equals(key, k). (There can be at most one such mapping.)

students.put Returns true if this map contains a mapping for the specified key. More formally, returns true if and only if this map contains a mapping for a key k such that Objects.equals(key, k). (There can be at most one such mapping.)
System.out.p

//search
System.out.p
int searchkey

System.out.p
int searchkey

if(students.containsKey(searchkey)){
    System.out.println("Students Map Contains key : "+ searchkey );
}
else{
    System.out.println("Students Map Doesn't Contains key : "+ searchkey );
}
```

Parameters:
• key key whose presence in this map is to be tested

Returns:
• true if this map contains a mapping for the specified key

Throws:
• ClassCastException - if the key is of an inappropriate type for this map (Collection##optional-restrictions optional)
• NullPointerException - if the specified key is null and this map does not permit null keys (Collection##optional-restrictions optional)

Remove Operation

removing the value By
key.

String java.util.Map.remove(Object key)

Removes the mapping for a key from this map if it is present (optional operation). More formally, if this map contains a mapping from key k to value v such that Objects.equals(key, k), that mapping is removed. (The map can contain at most one such mapping.)

Returns the value to which this map previously associated the key, or null if the map contained no mapping for the key.

If this map permits null values, then a return value of null does not necessarily indicate that the map contained no mapping for the key; it's also possible that the map explicitly mapped the key to null.

The map will not contain a mapping for the specified key once the call returns.

Parameters:
• key key whose mapping is to be removed from the map

Returns:
• the previous value associated with key, or null if there was no mapping for key

```
remove(removekey);
```

remove function returns the previous value associated with key, or null if there was no mapping for key

```
50 int removekey = 62;
51 System.out.println("Removing Student having Key "+ removekey );
52
53 System.out.println("Before Removing "+ students);
54
55 students.remove(removekey);
56
57 System.out.println("After Removing "+ students);
58
59
60
61
62
63
```

OUTPUT 63 DEBUG CONSOLE PORTS TERMINAL

{1=Veer, 2=Karan Shrigiri, 3=Rudresh, 25=Nandish, 300=Roshan, 62=Siddu}
Searching Operation
Students Map Contains key : 25
Removing Student having Key 62
Before Removing {1=Veer, 2=Karan Shrigiri, 3=Rudresh, 25=Nandish, 300=Roshan, 62=Siddu}
After Removing {1=Veer, 2=Karan Shrigiri, 3=Rudresh, 25=Nandish, 300=Roshan}
(base) kveeresh@KS-MacBook-Air-2 Computer-Science % cd /Users/kveeresh/Desktop/Compu

```

49
50     int removekey = 62;
51     System.out.println("Removing Student having Key "+ removekey );
52
53     System.out.println("Before Removing "+ students);
54
55     String removedVal  = students.remove(removekey); // return the Value which is Removed
56
57
58     System.out.println("Removed "+ removedVal);
59
60     System.out.println("After Removing "+ students);
61
62
63 }
64
65

```

```

Removing Student having Key 62
Before Removing {1=Veer, 2=Karan Shrigiri, 3=Rudresh, 25=Nandish, 300=Roshan, 62=Siddu}
Removed Siddu
After Removing {1=Veer, 2=Karan Shrigiri, 3=Rudresh, 25=Nandish, 300=Roshan}
(base) kveeresh@Ks-MacBook-Air-2 Computer-Science %
main* 59 2 0 Indexing completed. Java: Ready

```

Printing the HashMap Elements By keySet & EntrySet Help

Set<K> keySet()

Returns a **Set** view of the keys contained in this map.

Set<Integer> java.util.Map.keySet()

Returns a **Set** view of the keys contained in this map. The set is backed by the map, so changes to the map are reflected in the set, and vice-versa. If the map is modified while an iteration over the set is in progress (except through the iterator's own **remove** operation), the results of the iteration are undefined. The set supports element removal, which removes the corresponding mapping from the map, via the **Iterator.remove**, **Set.remove**, **removeAll**, **retainAll**, and **clear** operations. It does not support the **add** or **addAll** operations.

- **Returns:**
 - a set view of the keys contained in this map

Key Set ()

Entry Set

Set<Map.Entry<K, V>> entrySet()

Returns a **Set** view of the mappings contained in this map.

Set<Entry<Integer, String>> java.util.Map.entrySet()

Returns a **Set** view of the mappings contained in this map. The set is backed by the map, so changes to the map are reflected in the set, and vice-versa. If the map is modified while an iteration over the set is in progress (except through the iterator's **remove** operation, or through the **setValue** operation on a map entry returned by the iterator) the results of the iteration are undefined. The set supports element removal, which removes the corresponding mapping from the map, via the **Iterator.remove**, **Set.remove**, **removeAll**, **retainAll** and **clear** operations. It does not support the **add** or **addAll** operations.

- **Returns:**
 - a set view of the mappings contained in this map

```

61
62
63     System.out.println(x:"Printing Key Values of a Student HashMap");
64
65     System.out.println(x:"Using KeySet()");
66
67     for (Integer key : students.keySet()) {
68
69         System.out.println("(Key: "+key+" -> "+ "(Value: "+students.get(key)+"");
70     }
71
72
73     System.out.println(x:"Using EntrySet<K,V>");
74
75     for(Map.Entry<Integer,String> student : students.entrySet()){
76
77         System.out.println("(Key: "+student.getKey()+" -> "+ "(Value: "+student.getValue()+"");
78     }
79
80
81
82

```

Printing Key Values of a Student HashMap
Using KeySet()

```

(Key: 1) -> (Value: Veer)
(Key: 2) -> (Value: Karan Shrigiri)
(Key: 3) -> (Value: Rudresh)
(Key: 25) -> (Value: Nandish)
(Key: 300) -> (Value: Roshan)

```

Using EntrySet<K,V>

```

(Key: 1) -> (Value: Veer)
(Key: 2) -> (Value: Karan Shrigiri)
(Key: 3) -> (Value: Rudresh)
(Key: 25) -> (Value: Nandish)
(Key: 300) -> (Value: Roshan)

```

(base) kveeresh@Ks-MacBook-Air-2 Computer-Science %

Java HashMap

What is it? A `HashMap<K, V>` stores **pairs** like `rollNo` → `studentName`. You look up by **key** (e.g., roll number) and get the **value** (e.g., the student's name). It's **fast on average**: `put`, `get`, and `remove` are typically **$O(1)$** .

1) Why use a HashMap?

- **Fast lookups**: Find a value by key without scanning the whole list.
 - **Flexible keys/values**: Keys and values can be any objects.
 - **Real life analogy**: Think of a **college library index**. You don't scan every shelf; you go to a specific drawer using the index.
-

2) Quick picture: how it stores things

A `HashMap` keeps an **array of buckets** (think of them as **boxes**). The **key's hash** decides **which box** an entry goes into.

array index →	0	1	2	3	4	5	6	7	...	n-1
bucket pointer →		→ A →		B →			C →		...	

Legend: A/B/C = linked lists of entries (if multiple keys land in the same bucket)

- If two different keys go to the **same bucket**, they form a small **linked list** there (a **collision**).
 - Modern Java can convert long lists to a **tree** internally to stay fast, but as a beginner just remember: **on average it's $O(1)$** .
-

3) Minimal, very clear example

```
import java.util.HashMap;
import java.util.Map;

public class CollegeMapBasics {
    public static void main(String[] args) {
        // Create a map: rollNo (Integer) -> studentName (String)
        Map<Integer, String> students = new HashMap<>();

        // PUT: add (or replace) a key->value
        students.put(101, "Asha");
    }
}
```

```

students.put(102, "Ravi");
students.put(103, "Meera");

// GET: read value by key
System.out.println(students.get(102)); // "Ravi"

// UPDATE: putting same key replaces old value
students.put(102, "Ravi Kumar");
System.out.println(students.get(102)); // "Ravi Kumar"

// CONTAINS: check existence
System.out.println(students.containsKey(999)); // false

// REMOVE: delete by key
students.remove(103);

// SIZE & ITERATION: loop over entries (order is NOT guaranteed)
System.out.println("Total: " + students.size()); // e.g., 2
for (Map.Entry<Integer, String> e : students.entrySet()) {
    System.out.println(e.getKey() + " -> " + e.getValue());
}

// Safe default if key missing
String name = students.getOrDefault(555, "Unknown");
System.out.println(name); // "Unknown"
}
}

```

What just happened?

- put stores key→value.
- get fetches by key.
- Re-using a key **overwrites** the old value.
- **Order is not guaranteed** (don't expect insertion order).

4) How does HashMap find things so quickly?

1. It calls `key.hashCode()` to get a number.
2. It **maps** that number to a bucket index (like `index = hash % numberOfBuckets`).
3. It checks that bucket.
 - If the exact key is found (checked via `equals()`), it returns the value.
 - If there are multiple entries (collision), it walks that small list to find the match.

Tiny demo of collisions (conceptual):

Bucket 5:

```
[hash=..., key=101, value="Asha"] -> [hash=..., key=205, value="Pooja"] -> null
  ^ both keys happened to land in bucket 5, so we keep a small chain
```

5) Time complexity (Big-O)

Operation	Average time	Worst time (rare)	Notes
<code>put(key, value)</code>	O(1)	$O(\log n)$ / $O(n)^*$	*Java can tree-ify long chains to keep lookups near $O(\log n)$
<code>get(key)</code>	O(1)	$O(\log n)$ / $O(n)^*$	Depends on collisions; average is constant time
<code>remove(key)</code>	O(1)	$O(\log n)$ / $O(n)^*$	Similar reasoning
Iterate all entries	O(n)	$O(n)$	You visit every entry once

For beginner intuition: **Assume O(1) on average** for `put/get/remove`. The rare worst cases are handled internally to avoid huge slowdowns.

6) Important facts you'll use a lot

- **Nulls:** `HashMap` allows **1 null key** and **many null values**.
 - **Overwriting:** `put` with an existing key **replaces** the old value.
 - **No order:** `HashMap` does **not** guarantee iteration order.
 - **Fail-fast:** Changing the map while iterating (except via iterator's `remove`) can throw `ConcurrentModificationException`.
-

7) Custom keys (beginner-safe version)

If you use **your own class** as a **key**, you must define **logical equality** so that keys behave correctly.

```
import java.util.HashMap;
import java.util.Map;
import java.util.Objects;
```

```
// We'll key the map by StudentId (string). Two StudentId objects
```

```

// with the same id should be treated as the SAME key.
final class StudentId {
    private final String id; // keep it IMMUTABLE to avoid bugs

    StudentId(String id) {
        this.id = id;
    }

    // Two StudentId are equal if their 'id' text is equal
    @Override public boolean equals(Object o) {
        if (this == o) return true; // same object
        if (!(o instanceof StudentId)) return false;
        StudentId other = (StudentId) o;
        return Objects.equals(this.id, other.id);
    }

    // Must match equals: equal ids → equal hash codes
    @Override public int hashCode() {
        return Objects.hash(id);
    }

    @Override public String toString() { return id; }
}

public class CustomKeyBeginner {
    public static void main(String[] args) {
        Map<StudentId, String> names = new HashMap<>();

        names.put(new StudentId("21CS001"), "Asha");
        names.put(new StudentId("21CS001"), "Asha Updated"); // replaces previous (same log
        names.put(new StudentId("21CS002"), "Ravi");

        // Works because equals/hashCode say "21CS001" == "21CS001"
        System.out.println(names.get(new StudentId("21CS001"))); // "Asha Updated"
        System.out.println(names.size()); // 2
    }
}

```

Rule of thumb:

- Make the fields used in equals/hashCode **immutable**.
- If you change a key's field after insertion, the map may not find it again.

8) Resizing (what happens when it grows?)

- The map starts with some **capacity** (number of buckets).
- When it gets “too full” (controlled by **load factor**, default ~0.75), it **doubles** the buckets and **re-distributes** entries.
- This keeps buckets from getting crowded and maintains **O(1)** average time.

Concept sketch:

Before: capacity = 8, size 7 → resize

After: capacity = 16 (more buckets) → entries spread out again

You usually **don't** need to manage this manually, but if you know you'll store, say, **50,000 entries**, you can **pre-size**:

```
// Rough formula: initialCapacity  expectedSize / loadFactor
Map<Integer, String> big = new HashMap<>( (int)Math.ceil(50_000 / 0.75) );
```

9) A few very common patterns

A) Frequency count (word → count)

```
import java.util.*;

public class WordCount {
    public static void main(String[] args) {
        String[] words = {"java", "hashmap", "java", "map", "java", "code"};
        Map<String, Integer> freq = new HashMap<>();

        for (String w : words) {
            // If absent, default to 0; then add 1
            freq.put(w, freq.getOrDefault(w, 0) + 1);
        }
        System.out.println(freq); // {java=3, hashmap=1, map=1, code=1}
    }
}
```

B) Grouping by first letter

```
import java.util.*;

public class GroupByFirstLetter {
    public static void main(String[] args) {
        List<String> names = List.of("Asha", "Anil", "Ravi", "Ramesh", "Meera");
        Map<Character, List<String>> groups = new HashMap<>();
    }
}
```



```

    for (String name : names) {
        char first = name.charAt(0);
        // Make a new list if key not present
        groups.computeIfAbsent(first, k -> new ArrayList<>()).add(name);
    }
    System.out.println(groups); // {A=[Asha, Anil], R=[Ravi, Ramesh], M=[Meera]}
}

```

10) Quick checklist (for exams & interviews)

- Use **HashMap** when you need **fast** key→value lookups and **order doesn't matter**.
 - **Average O(1)** for put, get, remove.
 - **One null key** and **many null values** allowed.
 - **Equality of keys** is decided by `equals()` and **bucket choice** by `hashCode()`.
 - For **custom keys**, **override both** `equals()` and `hashCode()` and keep key fields **immutable**.
 - Iteration is **O(n)** and order is **not guaranteed**.
-

TL;DR

- **HashMap** = **fast dictionary** for Java.
- Think “**index → bucket → (maybe a tiny list)**” when visualizing.
- Use it for **fast lookups**, **counts**, and **grouping** by a key.